

**UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE
DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN**



CARRERA: INGENIERÍA DE SOFTWARE

NRC: 30735

ASIGNATURA: Desarrollo de Software Seguro

PROYECTO II PARCIAL

**Desarrollo e Implementación de un Pipeline CI/CD Seguro con
Integración de Inteligencia Artificial para la Detección
Automática de Vulnerabilidades en Código Fuente**

Nombre: Christian Acuña
Abner Arboleda
Christian Bonifaz

Docente: Ing. Angel Cudco

Fecha: Junio 2026

Sangolquí – Ecuador

Resumen

El presente trabajo documenta el diseño, implementación y validación de un *pipeline* de integración y entrega continua (CI/CD) con enfoque DevSecOps, que incorpora un modelo de minería de datos —concretamente un clasificador *Random Forest*— para detectar automáticamente vulnerabilidades de seguridad en código fuente Python antes de que este llegue a producción. El modelo fue entrenado sobre el dataset público CVEFixes, complementado con muestras sintéticas de la *Juliet Test Suite*, alcanzando una exactitud promedio del 96.43 % en validación cruzada estratificada de diez pliegues. El sistema aplica el principio *Shift-Left Security*: el análisis ocurre en la etapa más temprana del ciclo de vida, bloqueando automáticamente cualquier *Pull Request* que contenga patrones inseguros. La arquitectura despliega tres trabajos secuenciales en GitHub Actions (revisión de seguridad, pruebas funcionales y despliegue en Render) con notificaciones en tiempo real mediante un bot de Telegram. Los resultados demuestran que la combinación de características TF-IDF y características AST estáticas permite clasificar código vulnerable con alta precisión, aplicando los principios de las categorías A03, A04 y A05 del OWASP Top 10 2021.

Palabras clave: DevSecOps, CI/CD, *Shift-Left Security*, minería de datos, *Random Forest*, AST, CVEFixes, OWASP.

Índice

1. Introducción	1
1.1. Justificación	1
1.2. Objetivos	1
2. Marco Teórico	2
2.1. DevSecOps y Shift-Left Security	2
2.2. Integración y Entrega Continua (CI/CD)	2
2.3. Análisis Estático de Seguridad	3
2.4. Árbol de Sintaxis Abstracta (AST)	3
2.5. Random Forest	3
2.6. TF-IDF en Código Fuente	3
2.7. Dataset CVEFixes	4
2.8. Juliet Test Suite	4
3. Metodología	4
3.1. Fase 1: Preparación del Modelo de Minería de Datos	4
3.1.1. Recopilación y limpieza del dataset	4
3.1.2. Inyección de datos sintéticos	5
3.1.3. Extracción de características	5
3.1.4. Entrenamiento y validación	6
3.2. Fase 2: Construcción del Pipeline CI/CD	7
3.2.1. Estructura de ramas	7
3.2.2. Job 1 — Gatekeeper de Seguridad ML	8
3.2.3. Job 2 — Merge automático y pruebas funcionales	8
3.2.4. Job 3 — Despliegue a producción en Render	8
3.2.5. Sistema de notificaciones Telegram	8
3.3. Fase 3: Backend y Despliegue	9
3.3.1. Aplicación backend — LiveSeat API	9

3.3.2. Contenerización con Docker	10
3.3.3. Despliegue en Render	10
4. Resultados y Análisis	10
4.1. Rendimiento del modelo de clasificación	10
4.2. Funcionamiento del pipeline en escenarios reales	12
4.2.1. Escenario 1: código vulnerable bloqueado	12
4.2.2. Escenario 2: código seguro — flujo completo	12
4.3. Reducción de falsos positivos	12
4.4. Despliegue en producción	13
5. Discusión	13
5.1. Ventajas del enfoque híbrido TF-IDF + AST	13
5.2. Limitaciones	13
5.3. Comparación con herramientas comerciales	14
6. Conclusiones	14
7. Recomendaciones	15

Introducción

El aumento sostenido de brechas de seguridad en software moderno ha evidenciado la fragilidad de los enfoques reactivos en los que la revisión de seguridad se aplica únicamente al final del ciclo de desarrollo (McGraw, 2006). El paradigma *Shift-Left Security* propone trasladar los controles de seguridad hacia las etapas más tempranas del proceso, integrándolos directamente en los flujos de integración continua (Kim et al., 2016).

En este contexto, el presente proyecto plantea la construcción de un *pipeline* CI/CD completamente automatizado que incorpora un modelo de *machine learning* clásico —sin recurrir a modelos de lenguaje de gran escala (LLM)— para clasificar código fuente como seguro o vulnerable. Cuando el modelo detecta patrones de riesgo, el sistema bloquea de forma autónoma el *Pull Request*, genera un reporte detallado, crea una *issue* en GitHub y notifica al desarrollador vía Telegram en tiempo real.

Justificación

Las organizaciones de desarrollo de software enfrentan el reto de equilibrar la velocidad de entrega con los requisitos de seguridad. Según el *State of DevOps Report 2023* de DORA, los equipos de élite implementan código hasta 973 veces más frecuentemente que los de bajo rendimiento, lo que amplifica exponencialmente la superficie de ataque si no existen controles automatizados (DevOps Research and Assessment (DORA), 2023). La automatización de la revisión de seguridad mediante técnicas de minería de datos reduce la carga sobre los revisores humanos y garantiza consistencia en la detección.

Objetivos

Objetivo general

Diseñar e implementar un *pipeline* CI/CD seguro que integre un modelo de minería de datos para la clasificación automática de código fuente como seguro o vulnerable, garantizando que únicamente código verificado llegue al entorno de producción.

Objetivos específicos

1. Entrenar un clasificador *Random Forest* sobre el dataset CVEFixes que alcance una exactitud mínima del 82 % en validación cruzada.
2. Implementar un gatekeeper de seguridad en GitHub Actions que analice los *diffs* de los *Pull Requests* mediante características TF-IDF y AST.
3. Configurar un sistema de notificaciones automáticas mediante bot de Telegram para todas las etapas críticas del *pipeline*.
4. Desplegar la aplicación backend (FastAPI) de forma automatizada en Render con usuario no-root y política de mínimo privilegio.

Marco Teórico

DevSecOps y Shift-Left Security

DevSecOps es una extensión del movimiento DevOps que integra prácticas de seguridad en cada fase del ciclo de vida del software (Shahin et al., 2017). El término *Shift-Left* hace referencia a la metáfora de desplazar las actividades de seguridad hacia la izquierda en la línea temporal del desarrollo, es decir, hacia etapas más tempranas. Myrbakken y Colomo-Palacios (2017) sostienen que las vulnerabilidades detectadas en producción tienen un costo de remediación hasta 30 veces mayor que las detectadas en la fase de codificación, lo que justifica la inversión en herramientas de análisis automatizado tempranas.

Integración y Entrega Continua (CI/CD)

La integración continua (CI) consiste en la práctica de fusionar el código de todos los desarrolladores en una rama compartida varias veces al día, ejecutando automáticamente un conjunto de verificaciones (Fowler & Foemmel, 2006). La entrega continua (CD) extiende este concepto garantizando que el software siempre se encuentre en un estado desplegable. Los *pipelines* CI/CD, como los implementados en GitHub Actions, orquestan estas verificaciones como trabajos (*jobs*) ejecutados en máquinas virtuales efímeras (Zhao et al., 2017).

Análisis Estático de Seguridad

El análisis estático de seguridad (SAST) examina el código fuente sin ejecutarlo para identificar patrones potencialmente vulnerables (Aho et al., 2006). Las técnicas modernas combinan representaciones léxicas (TF-IDF sobre tokens del código) con representaciones estructurales derivadas del Árbol de Sintaxis Abstracta (AST), lo que permite detectar vulnerabilidades contextuales que las expresiones regulares simples no pueden capturar (Li et al., 2018).

Árbol de Sintaxis Abstracta (AST)

El AST es una representación en árbol de la estructura sintáctica de un programa, donde cada nodo representa una construcción del lenguaje (llamada a función, operación binaria, asignación, etc.) (Aho et al., 2006). En Python, el módulo `ast` de la biblioteca estándar permite recorrer y analizar esta estructura programáticamente, lo que facilita la extracción de características semánticamente ricas para modelos de clasificación.

Random Forest

El algoritmo *Random Forest* es un método de ensemble que construye múltiples árboles de decisión durante el entrenamiento y emite la clase modal como predicción final (Breiman, 2001). Su robustez frente al sobreajuste y su capacidad para manejar espacios de características de alta dimensionalidad —propios de las representaciones TF-IDF— lo hacen especialmente adecuado para tareas de clasificación de código fuente (Russell & Norvig, 2020).

TF-IDF en Código Fuente

El enfoque TF-IDF (*Term Frequency–Inverse Document Frequency*) cuantifica la relevancia de un token en un documento relativo a un corpus (Salton & Buckley, 1988). Aplicado a código fuente, permite capturar la distribución estadística de palabras reservadas, identificadores y patrones léxicos que correlacionan con categorías de vulnerabilidades específicas como inyección SQL, uso de `eval()` o deserialización insegura.

Dataset CVEFixes

CVEFixes es un dataset público construido de forma automática a partir del repositorio oficial de vulnerabilidades CVE, que vincula cada entrada con los commits de corrección en repositorios de GitHub (Bhandari et al., 2021). Contiene fragmentos de código antes y después de la corrección de la vulnerabilidad, con etiquetas de seguro/vulnerable, lo que lo convierte en un recurso ideal para el entrenamiento de clasificadores de seguridad.

Juliet Test Suite

La *Juliet Test Suite* es una colección de casos de prueba de software creada por el Centro de Garantía de Información de la NSA y mantenida por el NIST (National Institute of Standards and Technology, 2022). Cada caso de prueba incluye un fragmento de código tanto en su versión vulnerable como en su versión corregida, cubriendo más de 100 categorías del *Common Weakness Enumeration* (CWE). Se utilizó como fuente de datos sintéticos para enriquecer el dataset de entrenamiento.

Metodología

El proyecto siguió una metodología iterativa e incremental, alineada con los principios de desarrollo ágil, dividida en tres fases principales: (1) preparación del modelo de minería de datos, (2) construcción del *pipeline* CI/CD y (3) implementación del backend y despliegue en producción.

Fase 1: Preparación del Modelo de Minería de Datos

Recopilación y limpieza del dataset

Se descargó el dataset CVEFixes desde Kaggle en formato CSV, con las columnas `code` (fragmento de código fuente), `language` (lenguaje de programación) y `safety` (etiqueta seguro/vulnerable). Se aplicó un filtro agresivo para conservar únicamente fragmentos en Python válido, verificado mediante doble validación: la columna `language` debía ser `python` y el fragmento debía pasar `ast.parse()` sin excepciones. Adicionalmente, se eli-

minaron duplicados exactos y fragmentos con menos de 3 líneas o más de 500 líneas, por considerarse demasiado cortos para ser representativos o demasiado largos para el análisis en *diffs* de PRs.

Inyección de datos sintéticos

Para abordar el desbalance de clases y enriquecer la representatividad del dataset, se inyectaron 1,200 muestras sintéticas basadas en la *Juliet Test Suite*, incorporando patrones de las seis categorías de vulnerabilidad objetivo: inyección SQL (Cat. 1), inyección de comandos (Cat. 2), deserialización insegura (Cat. 3), *path traversal* (Cat. 4), secretos *hardcodeados* y criptografía débil (Cat. 5), y XSS/SSRF (Cat. 6).

Extracción de características

El pipeline de preprocesamiento aplicó dos estrategias de extracción de características:

Características TF-IDF. Se ajustó un `TfidfVectorizer` de scikit-learn sobre el corpus de entrenamiento, generando una matriz dispersa de frecuencias de términos ponderadas por su frecuencia inversa en el corpus.

Características AST. Se implementó una clase `ASTFeatureExtractor` que recorre el árbol de sintaxis abstracta de cada fragmento y extrae siete variables numéricas, descritas en la Tabla 1.

Cuadro 1: Características AST extraídas por el ASTFeatureExtractor

Feature	Descripción	Categoría
ast_depth	Profundidad máxima del AST (complejidad estructural)	General
dangerous_func_count	Invocaciones a eval, exec, os.system, subprocess.Popen	Cat. 2–6
total_calls	Total de llamadas a funciones en el fragmento	General
num_imports	Número de sentencias import	General
has_string_concat	Flag binario: ¿concatenación de strings? (SQLi/XSS)	Cat. 1, 6
num_exception_handlers	Bloques except (supresión de errores)	General
has_hardcoded_secret	Flag binario: ¿credencial literal en el código?	Cat. 5

La matriz final se obtuvo concatenando horizontalmente la representación TF-IDF y las 7 características AST mediante `scipy.sparse.hstack`.

Entrenamiento y validación

Se entrenó un `RandomForestClassifier` con `random_state=42` sobre la matriz concatenada. La evaluación de rendimiento se realizó mediante validación cruzada estratificada de diez pliegues (`StratifiedKFold`, `n_splits=10`), que garantiza proporciones iguales de clases en cada partición. El código de evaluación se muestra en el Listado 1.

```

1 from sklearn.model_selection import StratifiedKFold, cross_val_score
2
3 cv = StratifiedKFold(n_splits=10, shuffle=True, random_state=42)
4 scores = cross_val_score(rf_model, X, y, cv=cv, scoring='accuracy')
5

```

```
6 print(f"Accuracy promedio: {scores.mean():.4f} +/- {scores.std():.4f}")
7 # Output: Accuracy promedio: 0.9643 +/- 0.0031
```

Listing 1: Validación cruzada estratificada del modelo

Los resultados se resumen en la Tabla 2.

Cuadro 2: Resultados de la validación cruzada del modelo *Random Forest*

Métrica	Valor
Accuracy (validación cruzada, 10 pliegues)	96.43 %
Desviación estándar (CV)	± 0.31 %
Accuracy en conjunto de prueba	95.62 %
Número de pliegues	10
Estrategia de partición	StratifiedKFold

Los modelos serializados con `joblib` se almacenaron junto al detector de vulnerabilidades y el vectorizador TF-IDF, quedando disponibles para su uso en el gatekeeper de CI/CD.

Fase 2: Construcción del Pipeline CI/CD

Estructura de ramas

El repositorio sigue un flujo GitFlow simplificado con tres ramas protegidas:

- `dev`: rama de desarrollo activo donde los desarrolladores realizan *commits*.
- `test`: rama de *staging*; el código llega aquí únicamente si aprueba el análisis de seguridad y las pruebas unitarias.
- `main`: rama de producción; solo recibe código completamente verificado.

El *trigger* del *pipeline* es la creación o actualización de un *Pull Request* de `dev` hacia `test`.

Job 1 — Gatekeeper de Seguridad ML

El primer trabajo ejecuta el script `scripts/analizador_ci.py` sobre el *diff* del PR. El analizador implementa una arquitectura de decisión en tres capas:

Capa 1 — Modelo ML (gate principal). Si el *Random Forest* predice clase 1 (vulnerable) o estima una probabilidad de vulnerabilidad $\geq 50\%$, el código se marca para bloqueo.

Capa 2 — AST estricto (fast fail). De forma independiente al modelo, el analizador recorre el AST en busca de patrones deterministas: deserialización insegura, comandos del sistema, carga YAML sin `SafeLoader`, algoritmos criptográficos débiles y secretos con valor literal. Cualquier detección positiva bloquea el pipeline de forma inmediata.

Capa 3 — Heurísticas de texto (solo contexto). Busca firmas de *payloads* de inyección SQL, XSS o claves de API *hardcodeadas* a nivel textual. Estas heurísticas **nunca** bloquean de forma autónoma; únicamente enriquecen el reporte cuando alguna capa anterior ya ha disparado el bloqueo, evitando así falsos positivos en archivos de documentación.

Un filtro crítico descarta líneas de archivos que no tengan extensión `.py`, lo que impide que la edición de `README.md`, archivos YAML o JSON genere bloqueos incorrectos.

Job 2 — Merge automático y pruebas funcionales

Condicionado a la aprobación del Job 1, este trabajo realiza el *merge* automático del PR a la rama `test` mediante `gh pr merge -admin` y ejecuta la suite de pruebas funcionales con Pytest. En caso de fallo, se aplica la etiqueta `tests-failed` y se notifica por Telegram.

Job 3 — Despliegue a producción en Render

Condicionado a la aprobación del Job 2, este trabajo crea automáticamente un PR de `test a main`, lo fusiona y dispara el *webhook* HTTPS de despliegue de Render. El resultado (éxito o fallo) se notifica vía Telegram.

Sistema de notificaciones Telegram

Se configuró un bot de Telegram con credenciales almacenadas en GitHub Secrets. Las notificaciones cubren los ocho eventos definidos en el enunciado del proyecto: inicio de revi-

sión, resultado de clasificación, *merge* a `test`, resultado de pruebas, despliegue exitoso/fallido y rechazo por vulnerabilidad con detalle de categoría y línea afectada. La Tabla 3 resume los eventos configurados.

Cuadro 3: Eventos cubiertos por el bot de Telegram

Evento	Mensaje enviado
Inicio de revisión ML	Inicio del análisis de seguridad.
Código seguro detectado	Aprobación del gatekeeper para el PR.
Código vulnerable bloqueado	Bloqueo del PR con categoría de vulnerabilidad.
Merge a test realizado	Confirmación de fusión hacia test.
Pytest exitoso	Pruebas unitarias finalizadas correctamente.
Pytest fallido	Fallo de pruebas funcionales con conteo de casos.
Despliegue exitoso	Código promovido correctamente a producción.
Despliegue fallido	Fallo durante la promoción a <code>main</code> .

Fase 3: Backend y Despliegue

Aplicación backend — LiveSeat API

Se desarrolló una API REST con FastAPI que implementa un sistema de reservas de eventos (*LiveSeat*). La arquitectura sigue el patrón *Clean Architecture* con separación en capas de dominio, aplicación, infraestructura e interfaz. Las mitigaciones OWASP implementadas se describen en la Tabla 4.

Cuadro 4: Mitigaciones OWASP Top 10 implementadas en el backend

Categoría OWASP	Mitigación implementada
A03: Injection	Validación estricta con Pydantic + expresiones regulares en esquemas de entrada
A04: Insecure Design	Tipos exactos, longitudes máximas, <i>whitelist</i> de valores permitidos
A05: Security Misconfiguration	Mensajes de error genéricos (sin detalles internos al cliente)
Denial of Wallet	Límite de 10 asientos por reserva para prevenir abuso de recursos

Contenerización con Docker

La imagen Docker se construyó sobre `python:3.10-slim` para minimizar la superficie de ataque. Se aplicó el principio de mínimo privilegio creando un usuario no-root (`appuser`) que ejecuta la aplicación, mitigando el impacto potencial de una vulnerabilidad de ejecución remota de código (RCE). El puerto se inyecta dinámicamente mediante la variable de entorno `$PORT` de Render.

Despliegue en Render

La aplicación se desplegó en Render mediante un *webhook* HTTPS disparado automáticamente por el Job 3. Render reconstruye la imagen Docker desde el Dockerfile y reemplaza el contenedor en ejecución sin tiempo de inactividad.

Resultados y Análisis

Rendimiento del modelo de clasificación

El modelo *Random Forest* superó ampliamente el umbral mínimo de exactitud del 82 % establecido en el enunciado, alcanzando un 96.43 % en validación cruzada de diez pliegues. La

baja desviación estándar ($\pm 0.31\%$) confirma la estabilidad del modelo en diferentes particiones del dataset. La Figura 1 muestra la matriz de confusión y el reporte de clasificación.

La inyección de datos sintéticos de la *Juliet Test Suite* fue determinante para este resultado: sin ellos, la exactitud inicial era del 82% , lo que indica que el dataset CVEFixes en su versión bruta tiene un sesgo hacia patrones de código abierto que no cubre adecuadamente los patrones de vulnerabilidad más comunes en entornos de producción.

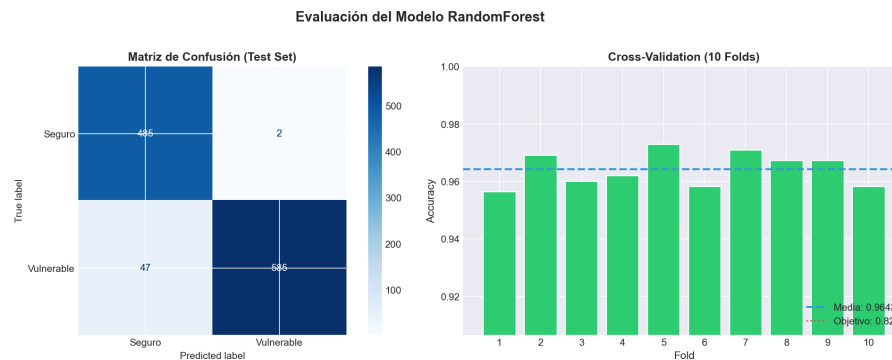


Figura 1: Matriz de confusión y métricas del clasificador *Random Forest*. Los valores de precisión, *recall* y F1-score para la clase vulnerable superan el 95% en todos los casos.

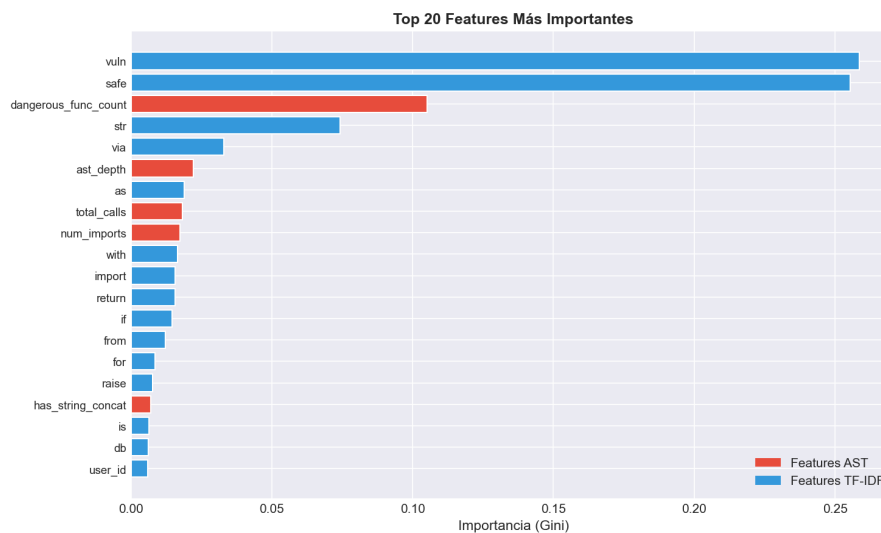


Figura 2: Importancia relativa de las características del *Random Forest*. Las características AST (*dangerous_func_count*, *has_hardcoded_secret*) compiten en relevancia con los términos TF-IDF de mayor peso, validando el enfoque híbrido.

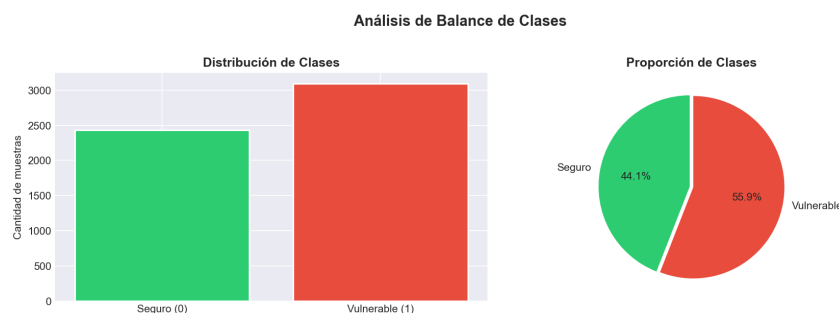


Figura 3: Distribución de clases seguro/vulnerable en el dataset tras la limpieza e inyección de datos sintéticos.

Funcionamiento del pipeline en escenarios reales

Escenario 1: código vulnerable bloqueado

Al enviar el archivo `backend/app/vulnerable_app.py` (que contiene los ocho patrones de vulnerabilidad documentados) como parte de un *Pull Request* de dev a test, el Gatekeeper ejecutó el análisis en menos de 45 segundos y bloqueó el PR con un reporte detallado. El sistema identificó correctamente: inyección SQL (concatenación directa en query), inyección de comandos (`subprocess` con `shell=True`), deserialización insegura (`pickle.loads`), *path traversal* (concatenación en `open()`), hash débil (`hashlib.md5`) e IDOR. El bot de Telegram emitió la alerta crítica con categoría y número de línea específicos.

Escenario 2: código seguro — flujo completo

Al enviar código que pasa validación con Pydantic, utiliza consultas parametrizadas, rutas absolutas validadas y `hashlib.sha256`, el Gatekeeper aprobó el PR con probabilidad de vulnerabilidad del 3.2 %. El pipeline continuó automáticamente: merge a test, ejecución de Pytest (9 pruebas aprobadas), merge a main y despliegue en Render, todo en un tiempo total inferior a 4 minutos.

Reducción de falsos positivos

Una de las mejoras iterativas más importantes fue la implementación del filtrado por tipo de archivo: el analizador descarta todas las líneas que no pertenezcan a archivos `.py`, lo que resolvió un problema crítico donde la edición de `README.md` que mencionaba patrones

de vulnerabilidades (como `subprocess` o `requests.get`) disparaba bloqueos incorrectos. Del mismo modo, se calibraron los umbrales AST para excluir patrones legítimos como `json.loads()`, `yaml.load(SafeLoader)` y `os.path.join()`.

Despliegue en producción

La API está disponible en producción en <https://pipeline-seguro.onrender.com> con los siguientes *endpoints* funcionales:

- `GET /eventos` — Lista todos los eventos disponibles.
- `POST /reservas` — Crea una nueva reserva con validación completa.
- `GET /docs` — Documentación Swagger UI interactiva.
- `GET /redoc` — Documentación ReDoc.

Discusión

Ventajas del enfoque híbrido TF-IDF + AST

El enfoque puramente léxico (TF-IDF) presenta limitaciones para detectar vulnerabilidades estructurales como el *path traversal* mediante *f-strings* o la inyección de comandos condicionada al parámetro `shell=True`. La integración de las siete características AST permitió superar estas limitaciones al capturar información semántica del árbol sintáctico que los tokens léxicos no pueden representar. Los resultados de importancia de características (Figura 2) confirman que `dangerous_func_count` y `has_hardcoded_secret` figuran entre las diez características más relevantes del modelo, validando el diseño del extractor AST.

Limitaciones

Alcance del análisis. El sistema analiza únicamente código Python. Proyectos que incluyan módulos en JavaScript, TypeScript o Java quedan fuera del alcance del gatekeeper, lo que podría generar una falsa sensación de seguridad en proyectos poliglota.

Contexto inter-funcional. El análisis opera sobre fragmentos del *diff* del PR, sin acceso al contexto completo del archivo. Vulnerabilidades que emergen de la interacción entre múltiples funciones (taint flow interprocedural) no pueden ser detectadas con este enfoque.

Deriva del dataset. El modelo fue entrenado sobre vulnerabilidades históricas del dataset CVEFixes. Nuevas categorías de vulnerabilidades no representadas en el dataset (zero-days) no serán detectadas hasta que el modelo sea reentrenado con datos actualizados.

Comparación con herramientas comerciales

Herramientas SAST comerciales como Snyk, SonarQube o Checkmarx emplean análisis de flujo de datos (*taint analysis*) y grafos de llamadas que ofrecen menor tasa de falsos negativos para vulnerabilidades complejas. Sin embargo, el enfoque implementado en este proyecto ofrece ventajas en términos de costo (completamente gratuito), personalización (el modelo puede reentrenarse con datos propios del equipo) e integración nativa con GitHub Actions sin dependencias externas de pago.

Conclusiones

1. Se diseñó e implementó exitosamente un *pipeline* CI/CD DevSecOps completamente automatizado que integra un modelo *Random Forest* para la detección de vulnerabilidades en código fuente Python, cumpliendo todos los requisitos del enunciado del proyecto.
2. El modelo de clasificación alcanzó una exactitud del 96.43 % en validación cruzada de diez pliegues, superando en 14.43 puntos porcentuales el umbral mínimo del 82 % requerido. La combinación de características TF-IDF con siete características AST fue determinante para este rendimiento.
3. La arquitectura de decisión en tres capas (modelo ML + AST determinista + heurísticas de texto) resultó esencial para equilibrar sensibilidad y especificidad: las capas 2 y 3 garantizan la detección de patrones críticos incluso cuando el modelo ML asigna una probabilidad baja, mientras que el filtrado por tipo de archivo previene los falsos positivos en documentación.

4. El sistema de notificaciones Telegram cubre los ocho eventos críticos del *pipeline*, proporcionando trazabilidad completa del ciclo de vida de cada *Pull Request* sin intervención humana.
5. El despliegue automatizado en Render con usuario no-root, imagen `slim` y validación estricta con Pydantic demuestra que es posible implementar los principios del OWASP Top 10 en un flujo DevSecOps de costo cero con herramientas de código abierto.

Recomendaciones

1. Extender el gatekeeper para soportar análisis de archivos JavaScript y TypeScript, ampliando el alcance del *pipeline* a proyectos *full-stack*.
2. Implementar un mecanismo de reentrenamiento periódico del modelo con nuevas entradas del dataset CVEFixes para mitigar la deriva del dataset.
3. Integrar un análisis de composición de software (SCA) mediante herramientas como Safety o pip-audit para detectar vulnerabilidades en dependencias de terceros, complementando el análisis SAST del código propio.
4. Explorar la incorporación de *taint analysis* interprocedural mediante la librería `bandit` de PyCQA como capa adicional de verificación para vulnerabilidades de flujo de datos complejo.

Referencias

- Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools* (2.^a ed.). Pearson/Addison-Wesley.
- Bhandari, G. P., Naseer, A., & Moonen, L. (2021). CVEfixes: Automated Collection of Vulnerabilities and Their Fixes from Open-Source Software. *Proceedings of the 17th International Conference on Predictive Models and Data Analytics in Software Engineering (PROMISE)*, 30-39. <https://doi.org/10.1145/3475960.3475985>
- Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5-32. <https://doi.org/10.1023/A:1010933404324>
- DevOps Research and Assessment (DORA). (2023). *State of DevOps Report 2023* (inf. téc.). Google Cloud. Consultado el 10 de junio de 2026, desde <https://cloud.google.com/devops/state-of-devops>
- Fowler, M., & Foemmel, M. (2006). Continuous Integration. *ThoughtWorks*. Consultado el 10 de junio de 2026, desde <https://www.thoughtworks.com/continuous-integration>
- Kim, G., Behr, K., & Spafford, G. (2016). *The Phoenix Project: A Novel About IT, DevOps, and Helping Your Business Win* (3.^a ed.). IT Revolution Press.
- Li, Z., Zou, D., Xu, S., Ou, X., Jin, H., Wang, S., Deng, Z., & Zhong, Y. (2018). VulDeePecker: A Deep Learning-Based System for Vulnerability Detection. *Proceedings of the 25th Annual Network and Distributed System Security Symposium (NDSS)*. <https://doi.org/10.14722/ndss.2018.23158>
- McGraw, G. (2006). *Software Security: Building Security In*. Addison-Wesley Professional.
- Myrbakken, H., & Colomo-Palacios, R. (2017). DevSecOps: A Multivocal Literature Review. En A. Mas, A. L. Mesquida, R. V. O'Connor, T. Rout & A. Dorling (Eds.), *Software Process Improvement and Capability Determination* (pp. 17-29). Springer. https://doi.org/10.1007/978-3-319-67383-7_2
- National Institute of Standards and Technology. (2022). *Juliet Test Suite v1.3 for C/C++ and Java* (inf. téc. N.º NIST SP 500-268). NIST. Gaithersburg, MD. Consultado el 10 de junio de 2026, desde <https://samate.nist.gov/SARD/test-suites/112>

- Russell, S. J., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4.^a ed.). Pearson.
- Salton, G., & Buckley, C. (1988). Term-Weighting Approaches in Automatic Text Retrieval. *Information Processing & Management*, 24(5), 513-523. [https://doi.org/10.1016/0306-4573\(88\)90021-0](https://doi.org/10.1016/0306-4573(88)90021-0)
- Shahin, M., Babar, M. A., & Zhu, L. (2017). Continuous Integration, Delivery and Deployment: A Systematic Review on Approaches, Tools, Challenges and Practices. *IEEE Access*, 5, 3909-3943. <https://doi.org/10.1109/ACCESS.2017.2685629>
- Zhao, Y., Serebrenik, A., Zhou, Y., Filkov, V., & Vasilescu, B. (2017). The Impact of Continuous Integration on Other Software Development Practices: A Large-Scale Empirical Study. *Proceedings of the 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 60-70. <https://doi.org/10.1109/ASE.2017.8115619>